

Conducting Wi-Fi Reconnaissance

Using the Aircrack-ng Suite

Sahithyan K. (230557T)

May 16, 2026

Contents

1	Introduction	2
1.1	Tools Used	2
2	Preparation	2
2.1	Installing Aircrack-ng	2
2.2	Checking Wireless Adapter Compatibility	2
2.3	Disabling Background Services	2
2.4	Enabling Monitor Mode	3
3	Activities, Results, and Interpretation	3
3.1	Passive Network Scanning	3
3.2	Targeted Packet Capture	4
3.3	Deauthentication Attack	5
3.3.1	Broadcast Deauthentication	5
3.3.2	Directed Deauthentication	5
3.3.3	Fake Authentication Attempt	5
3.4	WPA Handshake Capture	6
4	Summary and Conclusion	6
4.1	Recommendations	6

1. Introduction

Wi-Fi reconnaissance is the process of collecting information about nearby wireless networks before conducting security assessments or penetration testing activities. It helps identify wireless access points, security configurations, connected devices, and possible vulnerabilities within a wireless environment.

In this assignment, the Aircrack-ng suite was used to perform Wi-Fi reconnaissance. Aircrack-ng is a collection of tools used for wireless network monitoring, packet capturing, and security analysis.

1.1 Tools Used

- **airmon-ng** — enables monitor mode on the wireless adapter.
- **airodump-ng** — scans nearby wireless networks and captures packets.
- **aireplay-ng** — performs deauthentication attacks for capturing WPA/WPA2 handshake packets.

The reconnaissance process focused on gathering information such as SSIDs, BSSIDs, channels, encryption types, and signal strengths of nearby wireless networks.

2. Preparation

Before starting the Wi-Fi reconnaissance process, several preparations were completed to ensure proper functionality of the tools and wireless adapter.

Initially, I was planning to run the reconnaissance on my MacBook. However, the required functionality was not fully available due to hardware and driver limitations. The built-in Wi-Fi adapter did not support monitor mode and packet injection, which are necessary for **airodump-ng** and **aireplay-ng**. Because of this limitation, I used a friend's laptop running Ubuntu, which provided better support for Aircrack-ng and wireless adapter functionality.

2.1 Installing Aircrack-ng

The Aircrack-ng suite was installed on a Linux system using the package manager.

```
sudo apt install aircrack-ng
```

2.2 Checking Wireless Adapter Compatibility

Monitor mode and packet injection support were required. The wireless interface was verified using:

```
iwconfig
```

The detected wireless interface was **wlp2s0**.

2.3 Disabling Background Services

To prevent conflicts with the wireless interface, potentially interfering processes were stopped:

```
sudo airmon-ng check kill
```

The following processes were identified and killed:

PID	Name
1236	wpa_supplicant
1323558	avahi-daemon
1323562	avahi-daemon

2.4 Enabling Monitor Mode

To capture wireless traffic, monitor mode was enabled:

```
sudo airmon-ng start wlp2s0
```

After enabling monitor mode, the interface was renamed to `wlp2s0mon`.

3. Activities, Results, and Interpretation

3.1 Passive Network Scanning

Nearby wireless networks were scanned using `airodump-ng`:

```
sudo airodump-ng wlp2s0mon
```

This command displayed information about nearby access points and connected clients, including SSID, BSSID, channel, encryption type, signal strength, and associated clients. Six scanning sessions were conducted on 2026-05-16 between 12:27 and 12:54, totalling approximately 27 minutes of passive capture. A total of **39 unique access points** were detected.

A representative selection of the detected networks is shown below, ordered by signal strength:

SSID	BSSID	Ch.	Encryption	Signal
Thavanathan's iPhone	A6:8E:DC:82:EB:7B	6	WPA2	−47 dBm
Galaxy M0298c8	22:7C:A4:7D:D9:64	6	WPA2 / Open	−64 dBm
Boss	9E:02:9F:A9:99:85	1	WPA2	−66 dBm
Redmi A1+	BA:3D:EB:6D:12:5C	6	Open	−71 dBm
Dialog 4G	7C:11:CB:07:F3:03	6	WPA2	−76 dBm
Dialog 4G 777	98:A9:42:17:FE:C1	7	WPA2	−77 dBm
Dialog 4G 379	D8:42:F7:E2:21:7B	10	WPA2	−79 dBm
Redmi A5	4E:A8:A8:1D:FA:AF	1	WPA3 / WPA2	−84 dBm
EZVIZ_BA8983061	BA:4D:43:17:49:3D	3	WPA2 / WPA	−85 dBm
SLT-4G-EAFC	0C:8F:FF:28:14:7C	6	WPA2 / WPA	−91 dBm

Observations:

- **32 networks** used WPA2 encryption; **1 network** (Redmi A5) used WPA3.
- **4 networks** were fully open (no encryption), including the “Redmi A1+” hotspot.
- **7 networks** broadcast with a hidden SSID (empty ESSID field).
- Several networks advertised both WPA2 and the older WPA standard simultaneously, indicating mixed-mode or legacy configurations.

- Multiple mobile hotspots were visible, including devices named “Thavanathan’s iPhone”, “Galaxy M0298c8”, and “TimBaTTuK’s S25 Ultra”.
- IoT and surveillance devices were identifiable by their SSIDs: EZVIZ_ (security camera), DDPAI_ and 70mai_ (dash cameras), and three WIFIBRG_ bridge units.
- Vehicle-mounted access points were present: “Vezel-Dash-Cam” and “In_Vehicle_Wifi”.
- Up to 3 associated client devices were observed around a single access point during capture, making client tracking possible.

Interpretation:

The scan results illustrate how much a passive observer can learn without authenticating to any network. Four open networks expose all transmitted traffic in plaintext, leaving users vulnerable to credential theft and session hijacking. The 32 WPA2 networks provide reasonable protection with strong passphrases, but several ISP and IoT devices broadcast mixed WPA/WPA2 configurations, which reintroduce the weaker TKIP cipher and can be exploited. Only one network used WPA3, underscoring its slow real-world adoption. The seven hidden SSIDs offer no practical security benefit: access points still broadcast beacon frames and were captured trivially by `airodump-ng`. IoT devices (cameras, dash cameras, bridge units) advertise distinctive SSIDs and typically run outdated firmware, making them attractive entry points for network intrusion.

3.2 Targeted Packet Capture

A specific wireless network was selected for deeper packet capture:

```
sudo airodump-ng -c 6 --bssid XX:XX:XX:XX:XX:XX -w capture wlp2s0mon
```

- `-c` — specifies the channel number
- `--bssid` — specifies the target MAC address
- `-w` — writes captured packets to a file

The capture session ran from **12:56:33** to **12:57:12** (approximately 39 seconds). The following access point and client data were recorded:

SSID	BSSID	Ch.	Encryption	Signal	Beacons	Data
Thavanathan’s iPhone	A6:8E:DC:82:EB:7B	6	WPA2 (CCMP/PSK)	−78 dBm	322	1407

Two client devices were observed associated with this access point:

Client MAC	Signal	Packets	Note
24:EB:16:56:E1:CC	−76 dBm	3124	Directed deauth target (succeeded)
14:5A:FC:81:7E:1D	−37 dBm	1134	First deauth attempt (channel mismatch)

Interpretation:

Locking `airodump-ng` to a single channel and BSSID concentrates all captured frames on one target, enabling reliable client enumeration and packet accumulation within a very short window. The 1,407 data frames captured in under 40 seconds show that even brief targeted sessions yield enough material for further analysis.

3.3 Deauthentication Attack

A deauthentication attack was performed against an isolated private Wi-Fi access point (a personal mobile hotspot) to force connected clients to reconnect and generate WPA handshake packets. The target was selected because it was under personal control and no third-party devices were affected.

3.3.1 Broadcast Deauthentication

An initial broadcast deauthentication attack was sent to all clients associated with the target access point:

```
sudo aireplay-ng -0 5 -a A6:8E:DC:82:EB:7B wlp2s0mon
```

This sent five deauthentication frames to the broadcast address, requesting all connected clients to disconnect from the access point.

3.3.2 Directed Deauthentication

A directed attack targeting a specific client was then attempted. An initial attempt failed due to a channel mismatch — the monitor interface was locked to a different channel than the access point (channel 6):

```
sudo aireplay-ng -0 5 -a A6:8E:DC:82:EB:7B -c 14:5A:FC:81:7E:1D  
wlp2s0mon  
# wlp2s0mon is on channel 13, but the AP uses channel 6
```

Running `airmon-ng check kill` resolved the channel lock issue. A subsequent directed deauthentication against client `24:EB:16:56:E1:CC` succeeded, confirmed by the ACK counts reported by `aireplay-ng`:

```
sudo aireplay-ng -0 5 -a A6:8E:DC:82:EB:7B -c 24:EB:16:56:E1:CC  
wlp2s0mon  
# 12:46:10 Sending 64 directed DeAuth (code 7). STMAC:  
[24:EB:16:56:E1:CC] [63/67 ACKs]
```

A continuous attack (`-0 0`) was then run for approximately five minutes to observe sustained deauthentication behaviour. ACK counts varied across packets, indicating intermittent reconnection attempts by the client.

3.3.3 Fake Authentication Attempt

A fake authentication request was also sent to the access point to test association behaviour:

```
sudo aireplay-ng -1 0 -a A6:8E:DC:82:EB:7B -c 24:EB:16:56:E1:CC  
wlp2s0mon
```

The open-system authentication succeeded, but the subsequent association request was denied with status code 12 (*wrong ESSID or WPA*), confirming that WPA2 protection was enforced and association without valid credentials was not possible.

Interpretation:

The attack demonstrated a fundamental weakness in the 802.11 protocol: management frames such as deauthentication are unauthenticated by design, so any station can forge and send them

without possessing network credentials. The high ACK rates (63 out of 67) confirm that the frames were received and acted upon by the client. WPA3 networks with Management Frame Protection (MFP) are not vulnerable to this attack because their management frames are cryptographically signed; the absence of WPA3 across nearly all detected networks means this technique would work against the vast majority of access points observed.

3.4 WPA Handshake Capture

During reconnection triggered by the deauthentication attack, WPA handshake packets were captured successfully. The handshake notification appeared in the top-right corner of the `airodump-ng` window:

```
WPA Handshake: XX:XX:XX:XX:XX:XX
```

Interpretation:

Capturing the four-way WPA handshake is the prerequisite for offline passphrase cracking. Once the handshake is stored in a capture file, an attacker can attempt dictionary or brute-force attacks against it using tools such as `aircrack-ng` or `hashcat` without any further interaction with the target network. The risk to the network owner is therefore not limited to the moment of capture — a weak passphrase can be recovered long after the event. The collected information could potentially be used for unauthorised network access, password cracking, traffic monitoring, device tracking, and denial-of-service attacks.

4. Summary and Conclusion

This assignment demonstrated the process of conducting Wi-Fi reconnaissance using the Aircrack-ng suite. The tools `airmon-ng`, `airodump-ng`, and `aireplay-ng` were used to scan nearby wireless networks, gather information, and capture WPA/WPA2 handshake packets.

Across six scanning sessions conducted on 2026-05-16 (12:27–12:54), 39 unique access points were identified. Their SSIDs, BSSIDs, channels, encryption configurations, and signal strengths were collected passively without authenticating to any network. The activity also demonstrated how deauthentication attacks can force clients to reconnect and generate handshake packets for offline analysis.

The findings highlighted several real-world wireless security concerns: the presence of open hotspots, wide deployment of legacy mixed WPA/WPA2 configurations, near-absent WPA3 adoption, inadequately obscured IoT devices, and the fundamental insecurity of unauthenticated 802.11 management frames.

4.1 Recommendations

The following measures can improve wireless security:

- Use WPA2 or WPA3 encryption
- Avoid open wireless networks
- Use strong, unique passwords
- Disable WPS functionality
- Regularly update router firmware
- Monitor connected devices regularly

In conclusion, Wi-Fi reconnaissance is an important part of wireless security assessment because it helps identify vulnerabilities and security weaknesses before attackers can exploit them.